

Nested-ReFT: Efficient Reinforcement Learning for Large Language Model Fine-Tuning via Off-Policy Rollouts *

Maxime Heuillet^{1,2}, Yufei Cui³, Boxing Chen³, Audrey Durand^{1,2,4}, Prasanna Parthasarathi³

¹Université Laval (IID) ²Mila - Québec AI Institute

³Huawei Noah's Ark Lab (Montreal Research Center) ⁴Canada CIFAR AI Chair

Abstract

Advanced reasoning in LLMs on challenging domains like mathematical reasoning can be tackled using verifiable rewards based reinforced fine-tuning (ReFT). In standard ReFT frameworks, a behavior model generates multiple completions with answers per problem, for the answer to be then scored by a reward function. While such RL post-training methods demonstrate significant performance improvements across challenging reasoning domains, the computational cost of generating completions during training with multiple inference steps makes the training cost non-trivial. To address this, we draw inspiration from off-policy RL, and speculative decoding to introduce a novel ReFT framework, dubbed Nested-ReFT, where a subset of layers of the target model acts as the behavior model to generate off-policy completions during training. The behavior model configured with dynamic layer skipping per batch during training decreases the inference cost compared to the standard ReFT frameworks. Our theoretical analysis shows that Nested-ReFT yields unbiased gradient estimates with controlled variance. Our empirical analysis demonstrates improved computational efficiency measured as tokens/sec across multiple math reasoning benchmarks and model sizes. Additionally, we explore three variants of bias mitigation to minimize the off-policy bias in the gradient updates that allows for maintaining performance that matches the baseline ReFT performance.

Code — Under review.

Introduction

Large language models (LLMs) are increasingly capable at solving complex reasoning problems (Cobbe et al. 2021c). Such advances are enhanced by the LLMs ability to generate efficient chain-of-thoughts (CoT) through which the reasoning is broken down textually into intermediate logical steps (Wei et al. 2022). Improving generalization performance on the CoT completions of LLMs using post-training techniques has gained popularity; which leverages the range of possible CoT solutions during training (Kumar et al. 2025; Shao et al. 2024; Xie et al. 2024; Silver et al. 2016).

Modern LLM fine-tuning approaches leverage the range of possible CoT solutions by using reinforcement learning (RL) (Sutton et al. 2018). In RL-based fine-tuning, multiple

CoT completions are *rolled-out* from a *behavior* model, and they are then scored using a reward function that is another model (Cobbe et al. 2021a; Lightman et al. 2023) or a simple heuristic (Luong et al. 2024). The scored completions are then used to propagate the gradients back to fine-tune the *target* model. The rollout process augments significantly the amount of available data to fine-tune the target LLM.

RL-based fine-tuning with verifiable rewards corresponds to a specific fine-tuning problem called Reinforced Fine-tuning (ReFT) (Luong et al. 2024; Shao et al. 2024; Liu et al. 2025), which applies specifically to math and programming reasoning domains. As opposed to RL from human feedback (RLHF), a heuristic reward model can be used to verify and score the sampled completions instead of learning the reward model from human preferences (Rafailov et al. 2023).

Despite appealing performance gains, RL-based fine-tuning has a higher computational, and memory cost compared to supervised fine-tuning (SFT) (Luong et al. 2024). While ReFT circumvent the memory cost of storing a reward model, the computational cost of sampling multiple completions from a behavior model can be overwhelming (Kazemnejad et al. 2025; Shao et al. 2024). This completion cost can add up significantly to the compute cost of updating the parameters of the target model.

Practitioners currently sample 8 CoT completions per problem (von Werra et al. 2020), and the base setup consists in using as behavior model the same version of the target LLM (Luong et al. 2024; Shao et al. 2024). Although scaling up the number of CoT completions can lower the bias in the target model updates, it also adds a significant compute overhead. Therefore, there is a need to explore if the efficiency of the behavior model used for roll-outs can be improved. A broader impact would be to facilitate the generation of more completions to further improve the reasoning performance of LLMs.

Research question: *Is it possible to improve the computational efficiency of ReFT without compromising the performance of the fine-tuned target LLM?* We hypothesize that: i) given a target LLM to fine-tune, it is possible to perform roll-outs from a behavior model that has a lower computational cost than base formulations; ii) such low-cost roll-outs can be leveraged to update the target model with limited influence on performance.

* Manuscript submitted for review.

Contributions We introduce a novel ReFT framework, dubbed `Nested-ReFT`, where a subset of layers of the target model acts as the behavior model to generate off-policy roll-outs during training. The nested behavior model decreases the inference cost compared to the standard ReFT frameworks. Our theoretical analysis shows that `Nested-ReFT` yields unbiased gradient estimates with controlled variance. Our empirical analysis demonstrates improved computational efficiency across multiple math reasoning benchmarks and model sizes. Additionally, we explore three variants of bias mitigation to minimize the off-policyness in the gradient updates to maintain performance that matches the baseline ReFT performance.

Problem definition

Let $\mathcal{X}$ denote the space of possible prompts and $\mathcal{Y}$ denote the space of possible output sequences. Given a prompt $x_i \in \mathcal{X}$, an LLM encodes a generating policy π_θ , which defines a conditional probability distribution over output sequences $\hat{y}_i = (\hat{y}_{i,1}, \dots, \hat{y}_{i,L}) \in \mathcal{Y}$, where L is the number of tokens contained in the sequence. Let $\hat{y}_{i,<\ell}$ denote the tokens $(\hat{y}_{i,1}, \dots, \hat{y}_{i,\ell-1})$ in a sequence $\hat{y}_i$. The probability of sampling sequence $\hat{y}_i$ given a prompt x_i is defined in an autoregressive manner:

$$\pi_\theta(\hat{y}_i | x_i) = \prod_{\ell=1}^L \pi_\theta(\hat{y}_{i,\ell} | x_i, \hat{y}_{i,<\ell}),$$

where $\pi_\theta(\hat{y}_{i,\ell} | x_i, \hat{y}_{i,<\ell})$ is the probability of outputting token $\hat{y}_{i,\ell}$ given the prompt x_i and the previous tokens $\hat{y}_{i,<\ell}$.

Chain-of-thoughts and answers When applying LLMs to math reasoning, it is useful to distinguish chain-of-thought (CoT) sequences $\mathcal{Y}^{\text{cot}} \subset \mathcal{Y}$ from their value answers $\mathcal{Y}^{\text{val}} \subseteq \mathcal{Y}^{\text{cot}}$. The value is the exact solution to a math problem, while a CoT includes both the reasoning steps and the value. We assume access to a deterministic extraction function $v : \mathcal{Y}^{\text{cot}} \mapsto \mathcal{Y}^{\text{val}}$ that extracts value answers from CoTs.

Goal Consider a pretrained LLM, e.g., an open-sourced checkpoint from Hugging Face (Wolf et al. 2020). The objective is to fine-tune this LLM such as to maximize the performance on reasoning benchmarks. Consider benchmark k , denoted B_k . Let $(x_i, y_i^{k,\text{val}})$ denote the i -th example in benchmark B_k , where x_i is the prompt and $y_i^{k,\text{val}}$ is the target value answer. We compute the accuracy of the LLM answers $v(\hat{y}_i^k)$ on benchmark k as:

$$a_k = \frac{1}{|B_k|} \sum_{i=1}^{|B_k|} \mathbf{1}_{[v(\hat{y}_i^k) = y_i^{k,\text{val}}]}.$$

The goal is to maximize the overall performance, that is the average performance across a set of K benchmarks, defined as $a = \frac{1}{K} \sum_{k=1}^K a_k$.

Reinforced fine-tuning

Let θ_{ref} denote the parametrization of a pretrained LLM policy. Reinforced Fine-Tuning (ReFT) aims to further train $\pi_{\theta_{\text{ref}}}$ by leveraging reward feedback on a given dataset D , for S gradient steps contributing to E_{rft} epochs.

The dataset $D = (x_i, y_i^{\text{cot}})_{i=1}^{|D|}$ contains prompts describing math problems $x_i \in \mathcal{X}$ and their associated CoT solutions $y_i^{\text{cot}} \in \mathcal{Y}^{\text{cot}}$. Note that none of these math problems should be contained in the evaluation benchmarks.

Warm-up with SFT Prior to performing ReFT, it is common practice to perform E_{sft} epochs of supervised fine-tuning (SFT) on dataset D as a warm-up (Luong et al. 2024). Let θ_{sft} denote the parametrization of the resulting LLM policy, which serves as the initialization for the ReFT step.

Sample Generation via Behavior Policy Roll-outs Let π_θ denote the running target LLM policy that is being fine-tuned, and initialized with $\theta = \theta_{\text{sft}}$. During ReFT, a behavior LLM policy η_χ is used to perform roll-outs to generate solutions. For each prompt x in dataset D , the behavior model samples G solutions, $\{\hat{y}_g \sim \eta_\chi(\cdot | x)\}_{g=1}^G$. The samples are scored using a reward function $r : \mathcal{Y}^{\text{val}} \times \mathcal{Y}^{\text{val}} \rightarrow \mathbb{R}$ that compares the extracted value answer $v(\hat{y}_g)$ to the ground truth value associated to problem x , $r_g = r(v(\hat{y}_g), v(y^{\text{cot}}))$, $g = \{1 \dots G\}$. The scored samples are then used to update the target policy π_θ using a RL algorithm (e.g., GRPO, Shao et al. (2024)). The objective of the RL algorithm is to find parameters θ that maximize the expected reward.

Importance sampling Learning a target policy using rewards obtained from a separate behavior policy, is *off-policy RL*. Off-policy RL algorithms typically rely on *importance sampling* to account for the distribution difference between behavior and target policies. More specifically, rewards are reweighted using the importance sampling ratio:

$$h_{\text{base}}(\hat{y}, x; \pi_\theta, \eta_\chi) = \frac{\pi_\theta(\hat{y} | x)}{\eta_\chi(\hat{y} | x)} = \prod_{\ell=1}^L \frac{\pi_\theta(\hat{y}_\ell | x, \hat{y}_{<\ell})}{\eta_\chi(\hat{y}_\ell | x, \hat{y}_{<\ell})}.$$

The importance sampling ratio can suffer high variance, especially when the behavior and target policies diverge (Xie, Ma, and Wang 2019), which can negatively affect training quality. Variance reduction techniques are often employed to stabilize training (Munos et al. 2016; Metelli et al. 2020).

Current RL-fine-tuning configurations Most RL fine-tuning implementations (von Werra et al. 2020) and ReFT approaches (Luong et al. 2024) consider the behavior policy such that $\eta_\chi = \pi_{\theta_{\text{old}}}$, where θ_{old} corresponds to the target policy parametrization from the previous gradient step. Consequently, the importance sampling ratio is computed between the behavior policy $\pi_{\theta_{\text{old}}}$ and the target policy π_θ . However, there is no architectural difference between π_θ and $\pi_{\theta_{\text{old}}}$. This implies that the compute resources to generate samples from behavior policy is the same as the target policy.

In this work, we explore the possibility of generating samples from a behavior policy that has a lower inference cost, thus accelerating the inference with an increase in the off-policyness. Further, we bound the importance sampling ratio to reduce the variance in the updates for smooth training.

The Nested-ReFT framework

We introduce the `Nested-ReFT` framework, which instantiates behavior policy models nested in the target policy model. The nesting strategy is based on dynamic layer

skipping per batch throughout training, a protocol inspired from the field of speculative decoding (Xia et al. 2025a) and layer dropout (Fan, Grave, and Joulin 2019). While nesting improves sampling speed, it also increases the degree of off-policy-ness between the behavior and the target models as the architecture for the behavior model differs from the architecture of the target model. To mitigate the off-policy-ness, we then identify designs to decrease the variance of the importance sampling ratio (Munos et al. 2016). The Algorithm 1 summarizes the framework, with **purple** highlighting main differences with current ReFT. Though we experiment the framework with the popular GRPO algorithm (Shao et al. 2024), Nested-ReFT is agnostic to the RL algorithm and can be combined with every sampling based RL post-training techniques (Ahmadian et al. 2024; Ziegler et al. 2020).

Rollouts based on nested models

Consider the target model π_θ to fine-tune using ReFT and a behavior model η . We instantiate nested models with layer skipping. Layer skipping consists in selecting a set of layers that are not used (skipped) during the forward pass, acting like a short-cut.

Definition 1 (Set of transformer layers indices). The set of transformer layer indices in model η is noted $T_\eta = \{t_0, t_1, \dots, t_N\}$, where $|T_\eta| = N + 1$ denotes the total number of transformer layers.

Definition 2 (Set of valid layers indices). The *set of valid layers* indices $V_{\eta,b} = T_\eta \setminus \{t_0, \dots, t_b, t_{N-b}, \dots, t_N\}$ contains transformer layers indices within a distance b from the borders. The set of invalid layer indices is noted $\bar{V}_{\eta,b}$.

The set $\bar{V}_{\eta,b}$ points to layer indices that should not be skipped because they are too close to the inner and outer ends of the model (Elhoushi et al. 2024; Fan, Grave, and Joulin 2019). The inner and outer ends of deep learning models critically contribute to the generation of the model (van Aken et al. 2019). The total number of valid layers is $|V_{\eta,b}| = |T_\eta| - 2b$.

Consider now a ratio of layers to skip $x\%$, which is set by the user. The total number of layers skipped, noted U_x is

$$U_x = \begin{cases} \min(1, \text{ceil}(|T_\eta| \cdot x)) & \text{if } x > 0 \\ 0 & \text{otherwise.} \end{cases}$$

Note that the number of layers to skip is a function of T_η and not $V_{\eta,b}$ to maintain the number of skipped layers proportional to the original model size independently of b .

Definition 3 (Layer skipping module). Given a behavior model η , a skipping ratio $x\%$, and a border parameter b , a layer skipping module is a stochastic function $f_{x,b} : \eta \rightarrow [0, 1]^{|T_\eta|}$ that outputs a binary vector $\sigma = f_{x,b}(\eta)$, such that:

1. The number of indices flagged to be skipped is equal to U_x i.e. $\sum_{i=0}^{T_\eta} \sigma_i = U_x$,
2. The invalid indices are never flagged i.e. $\sum_{i=0}^{T_\eta} \sigma_i \mathbf{1}_{\{i \in \bar{V}_{b,\eta}\}} = 0$

Conditions 1) and 2) ensure that only valid layers are sampled. The U_x skipped layers are sampled i.i.d. and uniformly with probability $1/U_x$.

Ensemble of nested models throughout training At a given gradient step s in the ReFT training of S steps, the behavior model is defined such that $\eta_s = \pi_{\theta_{s-1}}$. This is aligned with prior works, where the behavior model corresponds to the old target model (commonly referred to as $\pi_{\theta_{\text{old}}}$). The nested behavior model η'_s is instantiated using $f_{x,b}(\eta_s)$. Throughout Nested-ReFT training, we obtain a stochastic ensemble of nested models $\mathcal{Z} = \{\eta'_s\}_{s=1}^S$. Setting the ratio $x = 0$ and $b = 0$, Nested-ReFT reduces to classical ReFT: $\eta'_s = \eta = \pi_{\theta_{\text{old}}}$.

Remarks Nested-ReFT is connected to the literature on *speculative decoding* (Xia et al. 2024), as the idea of using smaller nested models exists in that literature (Elhoushi et al. 2024; Zhang et al. 2024; Xia et al. 2025b). The main difference is that nesting was explored to accelerate inference at test time, while we instantiate nesting at train time. We will see in the theoretical analysis and empirical evaluation that transposing the nesting idea at train time brings a set of new unique challenges. Note that the proposed technique employs a depth wise nesting approach (layer skipping), but other width-wise nesting techniques of transformer layers exist as well (Narasimhan et al. 2025). This work also differs from Roux et al. (2025) where off-policy-ness is formulated as the collection of positive and negative samples to improve learning performance. In contrast, we aim to find a cost effective behavior policy while maintaining stability affected by the degree of off-policy-ness.

Algorithm 1: Nested-ReFT

Require: Target model π_θ (LLM), Dataset $\mathcal{D}$, reward function $R(x, y)$, skip ratio $x \in (0, 1)$, SFT epochs E_{sft} , RFT gradient steps S , border parameter b , choice of stabilization method $h_m(\cdot, \cdot)$, $m \in \{\text{base}, 1, \lambda\}$

- 1: **Step 1: Supervised Fine-Tuning (SFT)**
 - 2: **for** $e = 1$ **to** E_{sft} **do**
 - 3: Train π_θ on $(x, y^{\text{cot}}) \sim \mathcal{D}$ using cross-entropy loss
 - 4: **end for**
 - 5: **Step 2: Reinforced Fine-Tuning (ReFT)**
 - 6: **for** $s = 1$ **to** S **do**
 - 7: Sample batch of prompts
 - 8: **Set** $\eta_s = \pi_{\theta_{s-1}}$
 - 9: **Sample skip set with** $f_{b,x}(\eta_s)$
 - 10: **Deactivate layers in** η_s **using** $f_{b,x}(\eta_s)$ **to get** η'_s
 - 11: **Generate** G **samples for each** x **in batch using** η'_s
 - 12: Score samples using reward model
 - 13: **Compute stabilization** $h_m(\eta'_s, \pi_\theta)$
 - 14: **Update** π_{θ_s} **with rewards and** $h_m(\eta'_s, \pi_{\theta_s})$ **using RL objective**
 - 15: **Set** $\pi_{\theta_{s-1}} = \pi_{\theta_s}$
 - 16: **end for**
 - 17: **return** π_{θ_S}
-

Mitigation of increased off-policy-ness

We explore techniques to mitigate the notable high variance on the importance sampling ratio caused by high off-policy-ness. Specifically, we use simpler variations of the

base importance sampling ratio, summarized as $h_m(\cdot; \cdot)$, where $m \in \{\text{base}, 1, \lambda\}$.

- **Base approach:** The function is defined as the classical importance sampling ratio, corresponding to $h_{\text{base}}(\cdot, \cdot; \pi_{\theta_s}, \eta'_s)$. This corresponds to the base importance sampling implementation (Shao et al. 2024).
- **Practical approach:** The function $h_1(\cdot, \cdot; \pi_{\theta_s}, \eta'_s) = 1$. The motivation for this design choice is that the stochastic gradient descent is acknowledged as a powerful optimization protocol.
- **Retrace- λ approach** The function $h_\lambda(\cdot, \cdot; \pi_{\theta_s}, \eta'_s) = \lambda \min(1, h_{\text{base}}(\cdot, \cdot; \pi_{\theta_s}, \eta'_s))$. This approach is theoretically motivated following intuitions from Munos et al. (2016) who studied how to stabilize training under several degrees of off-policy in the traditional RL case. Emerging works follow such intuitions in the RL for LLMs setup (Roux et al. 2025).

Analysis of Nested-ReFT

Theoretical speed-up

Consider a behavior model η that contains T_η identical transformer layers. Each layer has a computational complexity that can be expressed as:

$$\mathcal{C}_{\text{layer}} = O(L^2 d + L d^2),$$

where L is the generated sequence length and d is the hidden dimension (i.e., width) of the layer (Vaswani et al. 2017).

Property 1 (Complexity with Layer Skipping). *Given a model η with T_η transformer layers, if we skip U_x layers, the computational complexity of a nested model η' is:*

$$\mathcal{C}_{\eta'} = O((T_\eta - U_x) \times \mathcal{C}_{\text{layer}}).$$

The complexity of the inference or the forward-pass of η' is reduced proportionally to the number of skipped layers U_x through the skip ratio $x\%$. Assuming fixed generation length L and hidden dimension d , the layer skipping achieves a linear complexity improvement.

Unbiased Convergence on the bounded off-policy

In reinforcement learning, we seek to optimize a policy π_θ using gradient-based updates. However, direct sampling from π_θ is not always feasible, so we employ a behavior model η for an off-policy update.

Unlike game RL environments (Brockman et al. 2016) where the states are independent from the policies, tasks like language generation involve generating the next token conditioned on a prompt and all the previous tokens. Therefore, the action is the prediction of the next token y_ℓ and the state is $s_\ell = (x, \hat{y}_{<\ell})$. Inducing exploration is non-trivial as it requires preserving the structural consistency of the state.

Hence, we opt for an ensemble of nested behavior policies $\mathcal{Z} = \{\eta'_i\}_{i=1}^{|\mathcal{Z}|}$ to generate off-policy updates of π_θ , where $|\mathcal{Z}| = S$ in Nested-ReFT. To estimate the policy gradient, we use importance sampling with the weight $h_{\text{base}}^i(y_\ell, s_\ell; \pi, \eta'_i)$. The behavior policy is selected uniformly from $\mathcal{Z}$, leading to an ensemble-weighted objective. Defining the mean behavior policy over the ensemble $\mathcal{Z}$ as:

$$\bar{\eta}_{\mathcal{Z}}(\hat{y}_\ell | s_\ell) = \frac{1}{|\mathcal{Z}|} \sum_{i=1}^{|\mathcal{Z}|} \eta'_i(\hat{y}_\ell | s_\ell), \quad (1)$$

the expected objective can be rewritten in terms of $\bar{\eta}_{\mathcal{Z}}$, ensuring stable updates as long as the importance weights remain bounded.

Suppose, we are interested in the policy gradient updates using advantage $(A_\ell^{\lambda, \gamma, \pi})$ (referred to A^π for brevity) estimation based on discounted λ -returns. Then, the advantage estimation for the target policy (π) is estimated as:

$$A^\pi(s, y) = Q^\pi(s, y) - V^\pi(s)$$

To estimate the policy gradient update we compute the derivative of the expected advantage objective ($\mathcal{J}$):

$$\mathcal{J} = \max \mathbb{E}_\pi [A^\pi] \quad (2)$$

$$\nabla \mathcal{J} = \nabla \mathbb{E}_\pi [A^\pi] \quad (3)$$

$$= \nabla \sum_{\ell=0}^L A^\pi(s_\ell, \hat{y}_\ell) \cdot \pi(\hat{y}_\ell | s_\ell) \quad (4)$$

$$= \nabla \sum_{\ell=0}^L A^\pi(s_\ell, \hat{y}_\ell) \cdot \pi(\hat{y}_\ell | s_\ell) \cdot \frac{\eta'_i(\hat{y}_\ell | s_\ell)}{\eta'_i(\hat{y}_\ell | s_\ell)} \quad (5)$$

$$= \nabla \sum_{\ell=0}^L A^\pi(s_\ell, \hat{y}_\ell) \cdot \eta'_i(\hat{y}_\ell | s_\ell) \cdot \frac{\pi(\hat{y}_\ell | s_\ell)}{\eta'_i(\hat{y}_\ell | s_\ell)} \quad (6)$$

$$= \nabla \sum_{\ell=0}^L A^\pi(s_\ell, \hat{y}_\ell) \cdot \eta'_i(\hat{y}_\ell | s_\ell) \cdot h_{\text{base}}^i(\hat{y}_\ell | s_\ell; \pi, \eta'_i) \quad (7)$$

Then, the objective for a $\eta'_i \in \mathcal{Z}$ can be re-written as,

$$\mathcal{J} = \max \mathbb{E}_{\eta'_i} [h_{\text{base}}^i \cdot A^{\eta'_i}]$$

Theorem 1. (Convergence of Policy Gradient with Ensemble Behavior Policies) *Let,*

1. *The importance weights $h_{\text{base}}^i(\hat{y}_\ell | s_\ell)$ are bounded,*
2. *The learning rate sequence over gradient steps s , α_s satisfies $\sum_s \alpha_s = \infty$ and $\sum_s \alpha_s^2 < \infty$, and*
3. *The behavior policy ensemble $\mathcal{Z}$ ensures sufficient exploration.*

With these assumptions, the policy gradient update using an ensemble of behavior policies converges to an optimum off-policy update from the expected advantage function weighted by the mean behavior policy $\bar{\eta}_{\mathcal{Z}}$.

Proof. The goal is to show that the using an ensemble of behavior policies for off-policy updates converges to the mean policy of the set, and the variance of the updates is controlled by a bounded measure on the importance ratio.

Step 1: Unbiasedness of the Gradient Estimate We start by rewriting the objective:

$$\mathcal{J}^Z = \max \mathbb{E}_{\eta'_i \sim \mathcal{Z}} [h_{\text{base}}^i \cdot A^{\eta'_i}]. \quad (8)$$

Since the behavior policy is selected uniformly at random, we can express this expectation as:

$$\mathbb{E}_{\eta'_i \sim \mathcal{Z}} [h_{\text{base}}^i \cdot A^{\eta'_i}] = \sum_{i=1}^{|Z|} p^i \cdot \mathbb{E}_{\eta'_i} [h_{\text{base}}^i \cdot A^{\eta'_i}], \quad (9)$$

where $p^i = \frac{1}{|Z|}$. Substituting the definition of w^i , we get:

$$\sum_{i=1}^{|Z|} \mathbb{E}_{\eta'_i} \left[\frac{h_{\text{base}}^i}{|Z|} A^{\eta'_i} \right]. \quad (10)$$

Rewriting as a sum over the sequence steps:

$$\sum_{i=1}^{|Z|} \sum_{\ell=0}^L h_{\text{base}}^i \cdot A^{\eta'_i}(s_\ell, \hat{y}_\ell) \cdot \frac{\eta'_i(\hat{y}_\ell | s_\ell)}{|Z|}. \quad (11)$$

By using the definition of the mean behavior policy, the equation simplifies to

$$\sum_{t=0}^T c \cdot A^{\bar{\eta}^Z}(s_t, \hat{y}_t) \cdot \bar{\eta}_Z(\hat{y}_t | s_t). \quad (12)$$

Since $\bar{\eta}_Z$ is the expectation over the behavior policies, the modified objective to update the target policy (π) with an ensemble of behavior policies $\mathcal{Z}$ is unbiased.

Step 2: Bounded Variance The importance sampling ratio influences the policy gradient update:

$$\text{Var} \left(h_{\text{base}}^i \cdot A^{\eta'_i} \right). \quad (13)$$

Since h_{base}^i is the ratio between the log-probabilities of both policies, the variance depends on how different η'_i is from π . Approaches like TB(λ), Retrace(λ), Off-policy Q(λ) have explored the variance minimization through bounding the off-policyyness of the behavior policy (Munos et al. 2016). The scale c can be bounded with h_1^i , or h_λ^i . The assumption that h_{base}^i is bounded ensures that:

$$\text{Var}(h_{\text{base}}^i) \leq c < \infty. \quad (14)$$

This ensures learning stability. $\square$

Experimental setup

We focus on the math reasoning task using five evaluation benchmarks, namely AIME2024 (Li et al. 2024), AMC (Li et al. 2024), MATH500 (Hendrycks et al. 2021), Minerva (Lewkowycz et al. 2022), and Olympiad (He et al. 2024). We consider two large language models *Qwen2.5-Math-Instruct* models (Yang et al. 2024) of sizes 1.5B and 7B. We consider three different datasets for fine-tuning, namely SVAMP (Patel, Bhattachamishra, and Goyal 2021), GSM8k (Cobbe et al. 2021b), and Math12k (Hendrycks et al. 2021).

Instances of Nested-ReFT We consider instances of Nested-ReFT with a proportion of skipped layers $x \in \{5\%, 10\%, 15\%\}$. Since both 1.5B and 7B LLMs have the same number of layers, their number of skipped layers is identical (see Table 1). We consider off-policyyness mitigation strategies using variance mitigation strategies on the importance sampling ratio h_m , with $m \in \{\text{base}, 1, \lambda\}$. The case h_1 is referred to as “practical” and h_λ as “Retrace- λ ” (Munos et al. 2016). The border parameter is set to $b = 1$, implying that only the first and last layers of the models are never skipped.

Model	N	W	Skipped layers at ratio x		
			5%	10%	15%
Qwen2.5-1.5B	28	1536	1	3	4
Qwen2.5-7B	28	3584	1	3	4

Table 1: Skipped layers for various ratios x on Qwen2.5-Math-Instruct. L = # of layers, W = hidden layer width.

Baselines For a given model, a baseline to any instance of Nested-ReFT corresponds to the model fine-tuned with Nested-ReFT at ratio $x = 0\%$, border $b = 0$ and mitigation method $m = \text{base}$. This instance corresponds to the model fine-tuned with ReFT using the base off-policyyness and importance sampling formulation from existing works (Shao et al. 2024; Luong et al. 2024). To our knowledge, there is no prior work that could fit as a fair baseline in the proposed new framework.

Training generation details We consider $E_{\text{sft}} = 2$ epochs for the SFT warm-up stage, similarly to (Luong et al. 2024). The β parameter of GRPO is set to 0, implying no KL penalty is used, following emerging evidence that the extra compute brought by the reference model is optional (Roux et al. 2025). The batch size is set to 16 for all models sizes, using gradient accumulation. We consider $S = 99$ gradient steps for ReFT, this corresponds to $E_{\text{rft}} = 1$, $E_{\text{rft}} = 0.11$, and $E_{\text{rft}} = 0.07$ for SVAMP, GSM8k and Math12k datasets, respectively. Fractions of epoch imply that a proportional subset of the shuffled dataset D is used for fine-tuning. This allows for fair cross-dataset and model comparisons. The prompts are formatted using a Qwen-chat template commonly used by practitioners (Liu et al. 2025). For the behavior model, we set the minimum and maximum length of the generated completions to 256 tokens. This implies that all the completions have equal length. For training, all the other parameters follow the default from GRPO TRL library (von Werra et al. 2020).

Evaluation generation details For evaluation, we use a math reasoning benchmark composed of 5 datasets (Liu et al. 2025). The temperature is set to 0.6, the top-p to 0.95 and the maximum number of tokens to 32k. We perform pass@K with $K = 1$, implying the model generates 1 response per problem. This corresponds to a strict setup as the model is only given one single chance to answer correctly.

Performance metrics and delta to the baseline To characterize reasoning performance at test-time, we report the

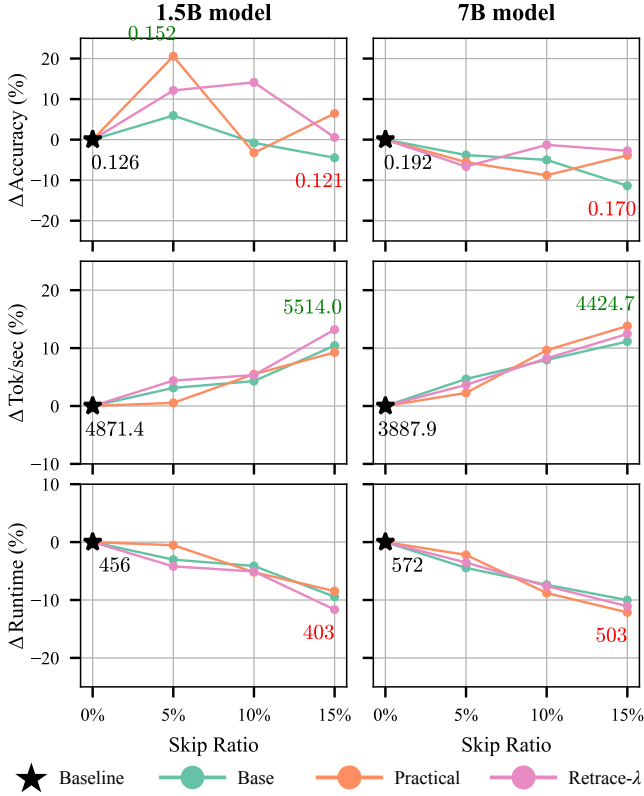


Figure 1: Fine-tuning on SVAMP. Red annotations indicate the smallest value, and Green annotations the largest value.

average accuracy on the 5 math benchmarks (Liu et al. 2025). To characterize compute efficiency gains at train-time, we report the token speed (total number of tokens processed divided by total runtime), and the total run time (expressed in seconds). We characterize Nested-ReFT run instances using the relative δ to the baseline, which is defined for any metric z as $\Delta(z) = 100 \cdot \frac{(z - z_{\text{baseline}})}{z_{\text{baseline}}}$, where the absolute delta is $\Delta_{\text{abs}}(z) = (z - z_{\text{baseline}})$.

Empirical Results

The setup comprises 12 distinct instances of Nested-ReFT per model, and 1 baseline. The experiment includes 2 models $\times$ 3 datasets $\times$ (12 + 1) instances = 78 experimental configurations. The results are displayed in Figures 1, 2, and 3. For Δ Accuracy (%) and Δ Tok/sec (%) the goal is to maximize the metric. For Δ Runtime (%), the goal is to minimize the metric. Table 2 references the absolute performance deltas.

Impact of off-policy roll-outs on performance

On Table 2, for 1.5B checkpoints, the mean absolute performance delta for best case Nested-ReFT is higher than for the worst case, indicating that the magnitude of the performance gains achieved with Nested-ReFT is bigger than the magnitude of the performance drops. However, on 7B checkpoints, the mean absolute performance delta for gains

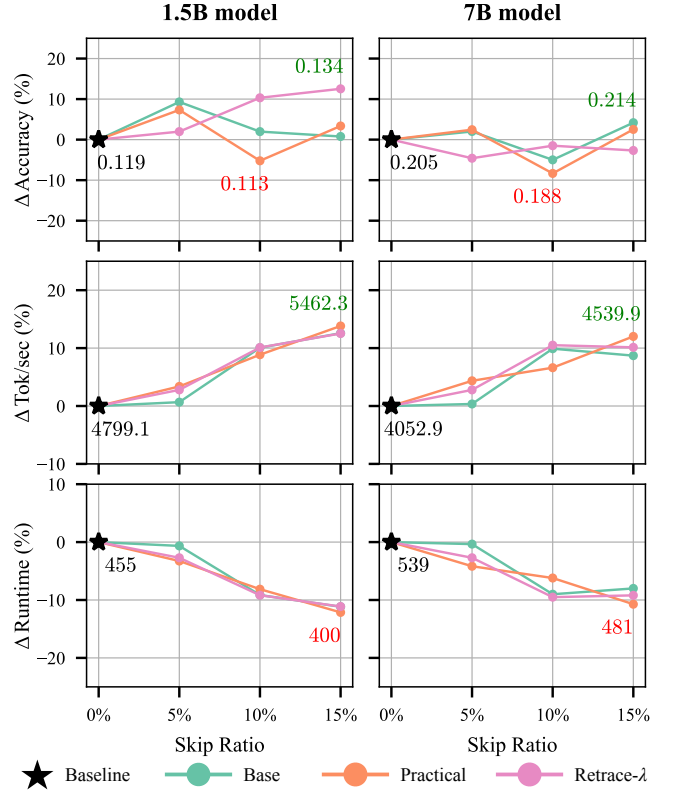


Figure 2: Fine-tuning on GSM8k. Red annotations indicate the smallest value, and Green annotations the largest value.

Model	Instance	SVAMP	GSM8k	Math12k	Δ_{abs}
1.5B	Best	+0.008	+0.015	+0.026	0.016
	Worst	-0.016	-0.006	-0.005	0.009
7B	Best	+0.022	+0.010	-0.003	0.009
	Worst	-0.070	-0.017	-0.022	0.036

Table 2: Best and worst observed deltas per dataset and model size. Δ_{abs} is the mean absolute change to baseline.

is smaller than that of the drops. In both cases, the worst and best performances imply at most ± 2.6 points variation from the baseline performance. These results showcasing minor performance fluctuations corroborate the hypothesis that off-policy generations using Nested-ReFT have limited influence on the performance on reasoning benchmarks. Importantly, we highlight that some instances of Nested-ReFT yield performance improvements over the baseline while involving the generation of samples on a smaller model, indicating that nested models can deliver similar or better effect as full models. In this research, the strategy to generate off-policy samples is heuristic. The results suggest that specialized learning strategies could improve further performance.

Effectiveness of the off-policyness mitigation strategy

We consider 3 off-policyness mitigation strategies, namely Base, Practical and Retrace- λ . We observe that

Retrace- λ displays the most stable performance across all models, fine-tuning datasets, and skipping ratios. Over the three datasets and two models (i.e. 6 configurations), the Base strategy achieves 1/6 best case count, 3/6 worst case count and 2/6 neutral count. The Practical strategy achieves 3/6 best case performance, 3/6 worst case, and 1/6 neutral count. This indicates that although Practical achieves peak performance, it is also unstable across configurations. The Retrace- λ strategy achieves 2/6 best case, 0/6 worst case and 4/6 neutral. These results indicate that Retrace- λ offers overall more stable performance compared to the Base and Practical mitigation strategies. The results further support the potential of Retrace- λ (Munos et al. 2016) to mitigate off-policyyness in the application of RL-based fine-tuning for LLMs. Other works (Roux et al. 2025) also point to Retrace- λ (Munos et al. 2016) to mitigate off-policyyness in LLMs, but the setting covered in Roux et al. (2025) assumes a delayed and possibly fixed behavior model (e.g. a reference model, or a frozen earlier version). In contrast, we cover a different setting where off-policyyness arises from a dynamic behavior model instantiated with a different architecture (nesting) than the target policy.

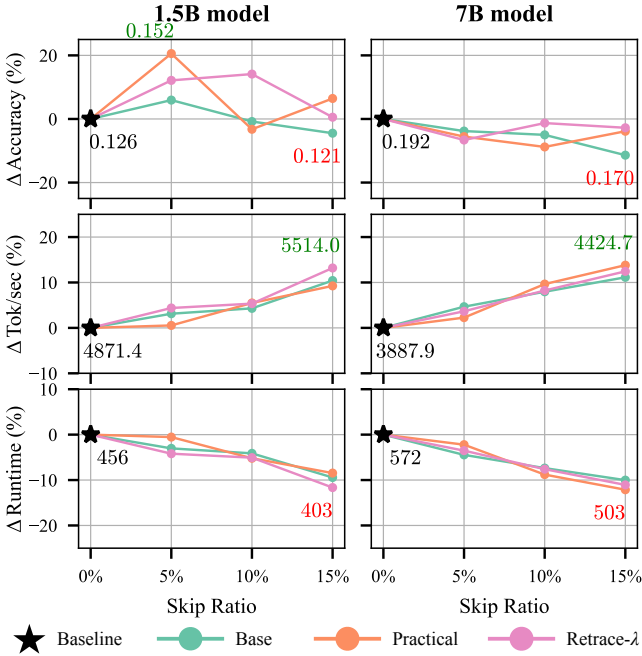


Figure 3: Fine-tuning on Math12k. Red annotations indicate the smallest value, and Green annotations the largest value.

Compute efficiency gains from off-policy roll-outs

Following the theoretical analysis on the complexity, the efficiency gains translate into linear trends on the total runtime, and on the token generation speed. This trend is observed in all the settings covered (see Figures 2, 3 and 1). Specifically, the efficiency gain on both metrics increases linearly with the ratio $x\%$ of skipped layers.

Related works

Parameter efficient fine-tuning (PEFT) Parameter efficient fine-tuning (Fu et al. 2023; Ding et al. 2023) consists in adapting only a subset of parameters. Low-rank (LoRA) adaptation (Hu et al. 2021) and its variants (Liu et al. 2024; Hayou, Ghosh, and Yu 2024) optimize training efficiency through the number of flops. Similarly, linear probing consists in optimizing the fine-tuning efficiency by restricting the parameter updates to the last layer of the LLM (Tomihari and Sato 2024). The proposed work is orthogonal to PEFT because we improve compute efficiency while all the parameters of the target LLM are updated.

Reinforcement learning for LLMs Luong et al. (2024); Kool, van Hoof, and Welling (2019); Schulman et al. (2017) propose frameworks but the limitation of the proposed frameworks is compute and memory cost. Dropping the reward model as in DPO (Rafailov et al. 2023) and KTO (Ethayarajh et al. 2024) can mitigate the memory overhead. However, in the specific problem of ReFT the reward model is a heuristic function, and the computational overhead is due to the behavior policy generating completions. To our knowledge, there is no work that aims to improve this completion generation cost in ReFT. More broadly in reinforcement fine-tuning (e.g. RLHF (Bai et al. 2022)) efficiency is addressed from a data selection perspective (Zhou et al. 2025; Shi et al. 2025) while the proposed framework addresses algorithmic variations for improved completion generation efficiency.

Conclusion

In this study, we explore the possibility of conducting off-policy RL fine-tuning. Specifically, we focus on the goal of achieving more compute efficient ReFT by instantiating the behavior model as nested instances of the target model. Our main conceptual contribution is to show that it is possible to increase the degree of off-policyyness in RL fine-tuning with minor influence on performance.

Emerging challenges for off-policy roll-outs Our controlled experiments show that it is possible to train smoothly, even when the degree of off-policyyness increases. The influence on performance of Nested-ReFT has limited impact on performance. These results are achieved for fixed size generation for the behavior model. However, an increasing number of LLMs can produce adaptive responses, either short for simple problems or long for complex problems. The interaction between generating completion off-policy through layer skipping and its influence on the completion length is on open research problem. Furthermore, the nesting strategy (e.g. layer skipping) may have non uniform interaction effects on the generation length depending on the dataset and model scale. This suggests that increasing off-policyyness with a learned strategy rather than a heuristic based approach based on layer skipping may handle the interactions more effectively.

References

- Ahmadian, A.; Cremer, C.; Gallé, M.; Fadaee, M.; Kreutzer, J.; Pietquin, O.; Üstün, A.; and Hooker, S. 2024. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs. *arXiv:2402.14740*.
- Bai, Y.; Jones, A.; Ndousse, K.; Askell, A.; Chen, A.; Das-Sarma, N.; Drain, D.; Fort, S.; Ganguli, D.; Henighan, T.; et al. 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*.
- Brockman, G.; Cheung, V.; Pettersson, L.; Schneider, J.; Schulman, J.; Tang, J.; and Zaremba, W. 2016. Openai gym. *arXiv preprint arXiv:1606.01540*.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021a. Training Verifiers to Solve Math Word Problems. *arXiv:2110.14168*.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021b. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; et al. 2021c. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Ding, N.; Qin, Y.; Yang, G.; Wei, F.; Yang, Z.; Su, Y.; Hu, S.; Chen, Y.; Chan, C.-M.; Chen, W.; et al. 2023. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature machine intelligence*, 5(3): 220–235.
- Elhoushi, M.; Shrivastava, A.; Liskovich, D.; Hosmer, B.; Wasti, B.; Lai, L.; Mahmoud, A.; Acun, B.; Agarwal, S.; Roman, A.; et al. 2024. LayerSkip: Enabling early exit inference and self-speculative decoding. *arXiv preprint arXiv:2404.16710*.
- Ethayarajh, K.; Xu, W.; Muennighoff, N.; Jurafsky, D.; and Kiela, D. 2024. Kto: Model alignment as prospect theoretic optimization. *arXiv preprint arXiv:2402.01306*.
- Fan, A.; Grave, E.; and Joulin, A. 2019. Reducing transformer depth on demand with structured dropout. *arXiv preprint arXiv:1909.11556*.
- Fu, Z.; Yang, H.; So, A. M.-C.; Lam, W.; Bing, L.; and Collier, N. 2023. On the effectiveness of parameter-efficient fine-tuning. In *Proceedings of the AAAI conference on artificial intelligence*.
- Hayou, S.; Ghosh, N.; and Yu, B. 2024. Lora+: Efficient low rank adaptation of large models. *arXiv preprint arXiv:2402.12354*.
- He, C.; Luo, R.; Bai, Y.; Hu, S.; Thai, Z. L.; Shen, J.; Hu, J.; Han, X.; Huang, Y.; Zhang, Y.; Liu, J.; Qi, L.; Liu, Z.; and Sun, M. 2024. OlympiadBench: A Challenging Benchmark for Promoting AGI with Olympiad-Level Bilingual Multimodal Scientific Problems. *arXiv:2402.14008*.
- Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. *arXiv:2103.03874*.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.
- Kazemnejad, A.; Aghajohari, M.; Portelance, E.; Sordoni, A.; Reddy, S.; Courville, A.; and Roux, N. L. 2025. VinePPO: Refining Credit Assignment in RL Training of LLMs. *arXiv:2410.01679*.
- Kool, W.; van Hoof, H.; and Welling, M. 2019. Buy 4 reinforcement samples, get a baseline for free! *arXiv*.
- Kumar, K.; Ashraf, T.; Thawakar, O.; Anwer, R. M.; Cholakal, H.; Shah, M.; Yang, M.-H.; Torr, P. H.; Khan, F. S.; and Khan, S. 2025. Llm post-training: A deep dive into reasoning large language models. *arXiv preprint arXiv:2502.21321*.
- Lewkowycz, A.; Andreassen, A.; Dohan, D.; Dyer, E.; Michalewski, H.; Ramasesh, V.; Slone, A.; Anil, C.; Schlag, I.; Gutman-Solo, T.; et al. 2022. Solving quantitative reasoning problems with language models. *Advances in neural information processing systems*, 35: 3843–3857.
- Li, J.; Beeching, E.; Tunstall, L.; Lipkin, B.; Soletskyi, R.; Huang, S.; Rasul, K.; Yu, L.; Jiang, A. Q.; Shen, Z.; et al. 2024. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. *Hugging Face repository*, 13(9): 9.
- Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2023. Let’s Verify Step by Step. *arXiv:2305.20050*.
- Liu, S.-Y.; Wang, C.-Y.; Yin, H.; Molchanov, P.; Wang, Y.-C. F.; Cheng, K.-T.; and Chen, M.-H. 2024. Dora: Weight-decomposed low-rank adaptation. In *Forty-first International Conference on Machine Learning*.
- Liu, Z.; Chen, C.; Li, W.; Qi, P.; Pang, T.; Du, C.; Lee, W. S.; and Lin, M. 2025. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*.
- Luong, T. Q.; Zhang, X.; Jie, Z.; Sun, P.; Jin, X.; and Li, H. 2024. Reft: Reasoning with reinforced fine-tuning. *arXiv preprint arXiv:2401.08967*, 3: 2.
- Metelli, A. M.; Papini, M.; Montali, N.; and Restelli, M. 2020. Importance sampling techniques for policy optimization. *Journal of Machine Learning Research*, 21(141): 1–75.
- Munos, R.; Stepleton, T.; Harutyunyan, A.; and Bellemare, M. G. 2016. Safe and Efficient Off-Policy Reinforcement Learning. *arXiv:1606.02647*.
- Narasimhan, H.; Jitkrittum, W.; Rawat, A. S.; Kim, S.; Gupta, N.; Menon, A. K.; and Kumar, S. 2025. Faster Cascades via Speculative Decoding. In *The Thirteenth International Conference on Learning Representations*.
- Patel, A.; Bhattamishra, S.; and Goyal, N. 2021. Are NLP Models really able to Solve Simple Math Word Problems? *arXiv:2103.07191*.
- Rafailov, R.; Sharma, A.; Mitchell, E.; Manning, C. D.; Ermon, S.; and Finn, C. 2023. Direct preference optimization:

- Your language model is secretly a reward model. *Advances in neural information processing systems*, 36: 53728–53741.
- Roux, N. L.; Bellemare, M. G.; Lebensold, J.; Bergeron, A.; Greaves, J.; Fr  chette, A.; Pelletier, C.; Thibodeau-Laufer, E.; Toth, S.; and Work, S. 2025. Tapered Off-Policy REINFORCE: Stable and efficient reinforcement learning for LLMs. *arXiv preprint arXiv:2503.14286*.
- Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y.; Wu, Y.; et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Shi, T.; Wu, Y.; Song, L.; Zhou, T.; and Zhao, J. 2025. Efficient Reinforcement Finetuning via Adaptive Curriculum Learning. *arXiv:2504.05520*.
- Silver, D.; et al. 2016. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587): 484–489.
- Sutton, R. S.; et al. 2018. *Reinforcement learning: An introduction*. MIT press.
- Tomihari, A.; and Sato, I. 2024. Understanding Linear Probing then Fine-tuning Language Models from NTK Perspective. *arXiv:2405.16747*.
- van Aken, B.; Winter, B.; L  ser, A.; and Gers, F. A. 2019. How Does BERT Answer Questions?: A Layer-Wise Analysis of Transformer Representations. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management, CIKM ’19*, 1823–1832. ACM.
- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser,   .; and Polosukhin, I. 2017. Attention is all you need. In *Proc. NeurIPS*, 30.
- von Werra, L.; Belkada, Y.; Tunstall, L.; Beeching, E.; Thrush, T.; Lambert, N.; Huang, S.; Rasul, K.; and Galou  dec, Q. 2020. TRL: Transformer Reinforcement Learning. <https://github.com/huggingface/trl>.
- Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q. V.; Zhou, D.; et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35: 24824–24837.
- Wolf, T.; Debut, L.; Sanh, V.; Chaumond, J.; Delangue, C.; Moi, A.; Cistac, P.; Rault, T.; Louf, R.; Funtowicz, M.; Davidson, J.; Shleifer, S.; von Platen, P.; Ma, C.; Jernite, Y.; Plu, J.; Xu, C.; Scao, T. L.; Gugger, S.; Drame, M.; Lhoest, Q.; and Rush, A. M. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45. Association for Computational Linguistics.
- Xia, H.; Du, C.; Li, Y.; Liu, Q.; and Li, W. 2025a. Tutorial Proposal: Speculative Decoding for Efficient LLM Inference. *arXiv:2503.00491*.
- Xia, H.; Li, Y.; Zhang, J.; Du, C.; and Li, W. 2025b. SWIFT: On-the-Fly Self-Speculative Decoding for LLM Inference Acceleration. *arXiv:2410.06916*.
- Xia, H.; Yang, Z.; Dong, Q.; Wang, P.; Li, Y.; Ge, T.; Liu, T.; Li, W.; and Sui, Z. 2024. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. *arXiv preprint arXiv:2401.07851*.
- Xie, T.; Ma, Y.; and Wang, Y.-X. 2019. Towards optimal off-policy evaluation for reinforcement learning with marginalized importance sampling. *Advances in neural information processing systems*, 32.
- Xie, Y.; Goyal, A.; Zheng, W.; Kan, M.-Y.; Lillicrap, T. P.; Kawaguchi, K.; and Shieh, M. 2024. Monte carlo tree search boosts reasoning via iterative preference learning. *arXiv preprint arXiv:2405.00451*.
- Yang, A.; Zhang, B.; Hui, B.; Gao, B.; Yu, B.; Li, C.; Liu, D.; Tu, J.; Zhou, J.; Lin, J.; Lu, K.; Xue, M.; Lin, R.; Liu, T.; Ren, X.; and Zhang, Z. 2024. Qwen2.5-Math Technical Report: Toward Mathematical Expert Model via Self-Improvement. *arXiv preprint arXiv:2409.12122*.
- Zhang, J.; Wang, J.; Li, H.; Shou, L.; Chen, K.; Chen, G.; and Mehrotra, S. 2024. Draft& Verify: Lossless Large Language Model Acceleration via Self-Speculative Decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 11263–11282. Association for Computational Linguistics.
- Zhou, Z.; Peng, A.; Li, Q.; Levine, S.; and Kumar, A. 2025. Efficient Online Reinforcement Learning Fine-Tuning Need Not Retain Offline Data. *arXiv:2412.07762*.
- Ziegler, D. M.; Stiennon, N.; Wu, J.; Brown, T. B.; Radford, A.; Amodei, D.; Christiano, P.; and Irving, G. 2020. Fine-Tuning Language Models from Human Preferences. *arXiv:1909.08593*.